

김민수

☎ 010-7714-9201 ✉ vibrio0102@gmail.com 🔗 linkedin 🏠 github

Core Skills

온프레미스 Kubernetes 위 MLOps 플랫폼 설계·구축·운영:

베어메탈 Kubernetes 위에 수집 → 레이크하우스 → 변환 → 계보 → 분산 학습 → 실험·모델 레지스트리까지 전 구간을 Pulumi(Python IaC)로 코드화해 혼자 구축하고 운영합니다. 10노드 · 상시 pod 235개 · 사내 서비스 23종을 프로덕션으로 돌리고 있고, 네트워크(Cilium) · 저장소(SeaweedFS) · 로그인(Dex OIDC → Kubernetes RBAC)까지 12개 스택을 하나도 빼놓지 않고 직접 짰습니다(스택 코드 변경의 93%). 그 뒤를 잇는 분산 학습 · 실험 기록 · 모델 저장소는 2026.08부터 붙이고 있습니다. 문제가 코드 밖에 있을 때는 랜선을 갈거나 스위치를 바꾸고 남는 RAM · NVMe를 쏘는 일까지 직접 합니다.

데이터 파이프라인과 레이크하우스:

Airflow 3.x + dbt(Cosmos) + Apache Iceberg 메달리온 파이프라인을 2026.03-07(5개월)에 설계·구현하고 DataHub에 계보를 자동 기록합니다(ETL 저장소 커밋의 83%, 공용 프레임워크의 93% 직접 작성). 이전에는 AWS EMR/PySpark 위에 대규모 클라우드 파이프라인을 운영했습니다.

측정으로 결정하는 아키텍처·트레이드오프 분석:

도구를 고르기 전에 잡니다. GPU 증설 요청을 10일 실측(가동률 5.3% · 대기 0건 · 유휴 점유 1,112 GPU-시간)으로 「구매 대신 회수」로 돌렸고, 네트워크·스토리지 재설계로 학습 데이터 전달을 25배 단축했으며, 연구 워크플로를 정량 조사해 재현성 계약을 설계했습니다. 비용·성능·운영 복잡도의 트레이드오프를 재서 고르는 방식은 PGVector 도입 등 이전 경력부터 같습니다.

AI 에이전트 상시 투입, 검증 장치로 받침:

AI 코딩 에이전트를 상시 투입해 1인 처리량을 팀 규모로 끌어올립니다(6개월 머지 PR 678건). 그 속도는 SDK 단위 테스트 719건, 태그/버전 불일치와 개인키 커밋을 막는 CI 게이트·pre-commit 혹은, 실패 기록 23건 · PRD 30건이 받칩니다. 이전에는 LangChain·임베딩 모델로 데이터 중복 제거 시스템과 챗봇 프로토타입을 개발했습니다.

풀스택 폭과 팀 리드:

iOS → 백엔드(FastAPI/Flask) → 프론트(Next.js) → 클라우드(AWS/GCP)를 거친 폭으로 데이터 엔지니어링 팀을 리드했고, 지금은 「도메인 ETL은 연구자가, 실행·계보·검증은 플랫폼이」 맡는 분담 구조를 설계해 정착시켰습니다.

Technical Skills

Languages: Python | Swift | JavaScript | Objective-C | Shell | SQL

Frameworks: PySpark | Flask | FastAPI | Airflow | Kafka | Langchain | OpenAI | ReactJS | NextJS | Bun

iOS: ReactorKit | Async Await | Alamofire | Snapkit | Crashlytics | Kingfisher | SceneKit | SPM | React-Native

Database: MariaDB | PostgreSQL | Firestore | Neo4J

OLAP: DuckDB | ClickHouse

DevOps: uv | Ubuntu | Docker | AWS (EC2, Lambda, S3, IAM, EMR, Route53, RDS Aurora), Firebase (Firestore, Storage), GCP

K8s Platform: Cilium | ClickHouse | PostgreSQL | KubeRay | MLflow | Dex | Grafana | Loki | Vector | DataHub

LLM: Langchain | LangGraph | OpenAI(Text Embedding) | RAG(Neo4J, PostgreSQL)

VectorStore: PGVector

Data Analytics: CytoscapeJS | SigmaJS

Career Summary

데이터 엔지니어 (약 3년): 클라우드(AWS) 환경에서 대규모 데이터 파이프라인을 설계, 구축, 최적화하는 데 깊은 경험을 보유. PySpark, Airflow, Vector DB 등을 활용하여 데이터 플랫폼 구축 및 MLOps 파이프라인 설계를 주도함.

백엔드 엔지니어 (누적 약 2-3년): Python(Flask, FastAPI) 기반의 API 서버 및 관리 시스템 개발 경험을 보유. WebSocket을 이용한 실시간 통신 및 Firebase 연동 등 다양한 백엔드 시스템을 구축함.

프론트엔드 및 클라이언트 (약 4년): Next.js 기반의 데이터 시각화 대시보드 및 BI 툴 개발 경험을 보유. iOS 개발자로 커리어를 시작하여 ReactorKit 기반의 반응형 아키텍처를 설계하고 네이티브 앱을 개발함.

Experiences

HUINNO

Dec 2025 - Present

Data Engineer

서울

- 사내 MLOps 파이프라인 전체를 처음부터 설계·구축·운영(수집 → 레이크하우스 → 변환 → 계보 → 분산 학습 → 실험·모델 레지스트리). 플랫폼 2인 팀에서 클러스터 구성 전 구간을 단독 수행 — 네트워크·스토리지·카탈로그·계보·오케스트레이션·분산 학습·신원까지 12개 Pulumi 스택 전부를 직접 만짐(스택 코드 변경의 93%, 6개 스택은 96% 이상). 6개월간 10노드·상시 pod 235개·사내 서비스 23종을 프로덕션 운영하며 머지 PR 678건·커밋 1,204건(92% 직접 작성)을 냈고, 베어메탈 Kubernetes를 Pulumi(Python IaC)로 전면 코드화해 전 구간을 재현 가능한 배포로 전환.
- ETL 파이프라인 전 갈래를 직접 설계·구현(2026.03-07, 5개월·별도 저장소, 커밋 1,599건 중 83% 직접 작성). 제품 공용 프레임워크 93%, CAST·MIMIC-IV·운영 유지보수 파이프라인 각 100%를 작성했고, 나머지 제품도 이 프레임워크 위에서 확장되도록 설계 — 도메인 ETL은 연구자가 옴고 실행·계보·검증·재시도는 플랫폼이 책임지는 분담 구조를 정착.
- [데이터 · 2026.03-07] 데이터셋 준비의 1인 의존을 제거. 기존에는 연구자가 데이터 과학자 한 명에게 요청 → 하드코딩된 스크립트를 SSH로 수 시간 실행 → 파일로 전달하는 구조라 담당자 부재 시 작업이 멈췄고 재현·계보·버전이 모두 없었다. Airflow 3.x + dbt(Cosmos) + Apache Iceberg로 Bronze/Silver/Gold 변환을 스케줄 자동 실행으로 바꾸고 DataHub에 테이블 단위 계보를 자동 기록해, 요청·대기 없이 연구자가 직접 조회하는 흐름으로 전환.
- [학습 · 2026.08-현재] 단일 GPU 수동 실행을 대체할 분산 학습 기반을 구축. KubeRay로 서로 다른 SKU가 섞인 GPU 10장을 단일 풀로 묶고, CUDA·PyTorch·ONNXRuntime 버전을 고정한 단일 워커 이미지와 사내 SDK 제출 API로 학습 환경을 일원화(자동 checkpoint·중단 지점 재개 지원). 도입 단계이며 연구 코드 이관이 남은 과제 — 이관을 막는 원인을 실측으로 규명한 것이 아래 두 항목.
- [설계 · 2026.08-현재] 기존 연구 워크플로를 실측 분석해 재현성 아키텍처를 설계. 연구 저장소를 정량 조사해 원격 브랜치 51개·출력이 저장된 노트북 셀 67~88%·실험 기록이 로컬 폴더에 고립됨을 규명하고, 원인이 도구 부재가 아니라 결과에 좌표가 없다는 점을 특정. 데이터셋에 버전·내용 해시·코드 커밋·환경 lock·시드를 실행 시점에 박는 재현성 계약(동결 도구·manifest 스키마·무결성 검증 CLI)을 설계·구현하고, MLflow 트래킹·모델 레지스트리(S3 아티팩트·SSO·DataHub 미러)를 그 좌표의 저장소로 배치.
- [기반] 학습 데이터 전달 시간을 25배 단축 — 네트워크·스토리지 재설계로 iperf3 실측 93.7 Mbps → 2.5 Gbps, 8 TB 기준 8.4일 → 7.8시간. SeaweedFS(S3 호환) 위 Iceberg 레이크하우스에 NVMe/HDD 티어 분리를 적용하고, PostgreSQL + pg_duckdb로 Iceberg를 ad-hoc SQL로 열어 pyiceberg 코드를 작성해야 하던 분석을 브라우저 SQL 한 줄로 단축.
- [하드웨어] 문제가 코드 밖에 있을 때도 직접 해결. 앞의 25배는 원인을 케이블까지 내려가 찾아낸 결과. 망가진 랜선을 찾아 갈고 필요한 스위치를 직접 골라 샀으며, 사내에 놓고 있던 RAM·NVMe를 모아 노드에 꽂아 사양을 올림. 선과 부품부터 Kubernetes 네트워크 부담까지 하드웨어든 소프트웨어든 가리지 않고 처리.
- [접근] 공유 비밀번호·무인증 접근을 단일 신원으로 통합. Dex(GitHub OIDC) + oauth2-proxy로 사내 서비스 23종에 SSO를 적용하고 같은 신원을 Kubernetes RBAC까지 연결. GPU 서버에 직접 SSH하던 개발 방식을 JupyterHub 브라우저 개발환경(개인 볼륨·egress 정책 격리·명령 한 줄 부트스트랩)으로 대체.
- [판단] 실측으로 GPU 증설 결정을 전환. 증설 검토 요청에 대해 DCGM·Prometheus로 10일간 측정한 결과 가동률 5.3%·대기 작업 0건이며 병목이 용량이 아니라 유휴 세션이 점유한 1,112 GPU-시간(점유 시간의 91%)임을 규명, 구매 대신 할당 회수로 방향을 전환.
- [방식] AI 코딩 에이전트를 상시 투입해 1인 처리량을 팀 규모로 끌어올리되, 속도를 검증 장치로 받침. 위 산출물을 6개월에 내면서 SDK 단위 테스트 719건, 태그/버전 불일치·개인키 커밋을 막는 CI 게이트와 pre-commit 훅, 사고 재발을 막는 실패 기록 23건·PRD 30건을 함께 축적.

Cellkey

Jul 2025 - Dec 2025

Manager, Data Engineer

서초구, 서울

- Apache Airflow (3.x)를 활용하여 데이터 정제 Pipeline을 설계 및 로컬 환경에 배포함.
- AWS Lambda, Batch, Step Functions 등 주요 리소스를 유지보수 및 최적화하여 확장 가능한 데이터 워크플로우를 구축함.

Netmelon

Mar 2025 - Jun 2025

Back-End Engineer

Freelance

- Android, iOS In App Purchase API (구독, 구독갱신, 구독취소) 구현
- Google Cloud Run 에 Flask 기반 Docker Containerized Web server 배포
- Firebase Resource들에 대해 Ownership을 확인하는 기능 구현
- LLM Driven API Routes, API Services, Functions Unit test cases 작성
- Firebase Firestore, Firestorage 운용
- 레거시 코드 리팩토링

(주) Eazel

Team Lead, AI Labs

Jun 2022 - Sep 2024

중구, 서울

- 미술 시장 ETL 데이터 파이프라인 총괄 개발 및 운영: 팀 리드로서 아키텍처 설계부터 배포, 모니터링, 성능 최적화 주도
- Flask/NextJS 기반 대시보드 구축
- 옥션 데이터 스크래퍼 개발 및 운영
- Airflow를 이용하여 주기적인 데이터 수집, 정제 및 적재 파이프라인 구현
- AWS EMR의 PySpark Application을 이용하여 데이터 정제, 적재 파이프라인 구현
- OpenAI/Facebook Embedding 모델 활용 데이터 중복 제거 시스템 개발 및 정확도/속도 개선 (벡터 DB 적용 등)
- AWS 서비스 관리 (EC2, IAM, S3, Route53, RDS, EMR)
- Github Action 을 이용하여 Continuous Distribution 구현
- LangChain, Streamlit을 이용하여 Chatbot Prototype 구현
- Chatbot을 위한 Neo4J Cypher query Tool 구현 주도, PostgreSQL Query Tool 구현 주도.
- LangChain, Information Extracting Prompt를 사용하여 데이터 스크래핑 구현

(주) Eazel

MLOps Engineer, MLOps

Oct 2021 - Jun 2022

중구, 서울

- 데이터 분석부터 모델링, 검증까지 End-to-End 프로세스 경험 (HuggingFace NLP 등 활용)
- 인스타그램 데이터 스크래퍼 개발 및 운영
- Flask/NextJS/CytoscapeJS 기반 소셜 관계망 분석 대시보드 개발 및 인사이트 도출 (Series A 투자 유치 기여)
- DVC를 이용하여 ML Lifecycle 구현
- PostgreSQL 사용하여 데이터 관리

(주) Eazel

iOS Developer, iOS Team

Jan 2019 - Oct 2021

강남, 서울

- iOS 앱 성능 및 확장성을 고려한 API 클라이언트 및 연동 백엔드 로직 설계 주도
- Github Action 으로 CI/CD 도입
- ReactorKit 기반 Reactive Programming 도입 및 아키텍처 설계 주도, 앱 반응성 및 유지보수성 향상

(주) Finple

Software Engineer, DevOps

Jan 2018 - Nov 2018

영등포, 서울

- 하이브리드 iOS, Android 앱 유지보수
- NodeJS 기반 데이터 스크래퍼 개발

Projects

(주) Eazel

Jan 2019 - Sep 2024

DataEngineer, AI Labs

Jun 2022 - Sep 2024

중구, 서울

Price Database 서비스를 위한 데이터 파이프라인 개발

Jan 2023 - Oct 2023

- 개발 리소스 절약을 위해 단일 AWS Aurora RDS 사용하여 데이터 파이프라인 구축
- AWS EMR의 PySpark Application 을 이용하여 데이터 가공 및 적재
- 분산 처리를 위한 파티셔닝 및 데이터 최적화
- 데이터 플랫폼 (Price Database/MLOps) 총괄 설계 및 구축으로 Series B 투자 유치에 핵심 기여
- 주도적으로 구축한 대용량 처리가 가능한 데이터 파이프라인 구축
- [Project Detail >>](#)

미술시장 데이터 스크래퍼 개발

Jan 2023 - Feb 2023

- requests, BeautifulSoup4 을 이용하여 데이터 스크래퍼 개발
- MultiThreading 을 활용하여 고속으로 리소스를 업로드하는 스크래퍼 개발
- Langchain 의 Information Extraction Prompt를 이용하여 정형 데이터 스크래핑 기술 개발
- Swift로 작성하여 iOS 환경에서 동작하는 소셜 관계 수집 스크래퍼 개발
- boto3 이용하여 다중 스크래퍼 인스턴스 관리하여 50개의 스크래퍼 인스턴스 동시 관리
- WebSocket을 이용하여 스크래퍼 관리 서버와 연결하여 스크래퍼 인스턴스 관리
- 동적(Selenium, PlayWright), 정적(Public/Private API) 데이터 스크래퍼 개발 및 운용
- [Project Detail >>](#)

데이터 스크래퍼 중앙 관리 서버 개발

Jan 2023 - Feb 2023

- Flask를 이용하여 스크래퍼들을 중앙 관리하기 위한 API endpoints 개발
- WebSocket을 이용하여 스크래퍼와 연결하여 스크래퍼에게 타겟에 대한 정보를 넘겨줘 스크래핑
- SHA-256 암호화를 이용하여 Login Credential 전달
- Flask Blueprint를 이용하여 API Endpoints 개발
- Nginx를 이용하여 reverse proxy 구현
- Rout53과 Certbot을 이용하여 HTTPS 인증서 관리
- **중앙 관리 용이한 데이터 스크래퍼 서버 개발 및 운용**
- [Project Detail >>](#)

BI 툴 개발

Nov 2023 - Nov 2023

- Flask와 NextJS 이용하여 BI툴을 위한 서버 개발
- 소셜 관계 데이터를 전달해주는 API 개발
- Directed Graph 형태를 갖는 소셜 관계를 보여주기 위한 BI Page를 Cytoscape와 SigmaJS를 이용하여 개발
- 데이터 라벨링을 위한 API 개발
- NextJS와 Carbon Design System을 이용하여 Components 개발
- Rout53과 Certbot을 이용하여 HTTPS 인증서 관리
- Docker Compose를 이용하여 EC2 Instance(Ubuntu 22.04)에 배포
- **FullStack BI(Nginx, Nextjs, Flask) 툴 개발 및 운용**
- [Project Detail >>](#)

Airflow 서비스 배포 & DAG 작성

Jan 2023 - Feb 2024

- Airflow의 PythonOperator 를 이용하여 스크래퍼 중앙 관리 서버와 통신하여 스크래핑 스케줄링 DAG 작성
- boto3를 이용하여 AWS EMR 서비스를 이용하여 데이터 정제 및 적재 시케줄링 DAG 작성
- Sequential Executor를 이용하여 DAG 운영
- Rout53과 Certbot을 이용하여 HTTPS 인증서 관리
- Docker Compose를 이용하여 EC2 Instance(Ubuntu 22.04) 에 서비스 배포
- **Airflow DAG를 통한 주기적인 데이터 스크래핑 스케줄링 및 PySpark를 이용하여 데이터 정제**
- [Project Detail >>](#)

데이터 중복 제거 기능 개발

Nov 2023 - Feb 2024

- ML Research Engineer와 협업하여 Faiss-CPU를 이용하여 IVFFlat Index 생성하여 Embedding Vector의 거리를 계산하여 중복 제거 시스템 개발
- Facebook의 SSCD 모델 사용하여 Image Embedding Vector 추출
- OpenAI의 Text Embedding 모델 Ada를 사용하여 Text Embedding Vector 추출
- Postgres의 PGVector 0.5.0 Extension을 이용하여 Index 생성 오버헤드를 50% 줄임
- PGVector의 IVFFlat, HNSW Index 생성
- 기존 Numpy.array를 pgvector::vector로 데이터 변경
- **신뢰도 높은 데이터를 위한 Embedding 기반 데이터 정제**
- [Project Detail >>](#)

효율적인 데이터 관리를 위해 데이터 웨어하우스 스키마 설계

Feb 2024 - Sep 2024

- 여러 데이터 소스에서 수집되어 발생하는 중복 데이터들을 효율적으로 관리하기 위해 Single Source of Truth 테이블을 포함한 데이터웨어하우스를 DBDiagram으로 설계
- 기존 RDS를 데이터소스 및 데이터 웨어하우스 저장소로 사용하여 발생된 비용을 줄이기 위해 S3를 데이터레이크로 사용하는 데이터웨어하우스 스키마로 설계
- DataSource, DataWarehouse, DataMart와 SSoT 스키마 설계, 약 50개 분량의 테이블을 설계
- **비용 절감 및 신뢰도 높은 데이터 관리를 위한 데이터 파이프라인 구조 재설계 주도**
- [Project Detail >>](#)

AWS Service 관리

- IAM Role 기반 유저 별 서비스 접근 제어
- 비용 절약을 위해 EC2 x86 인스턴스 타입을 arm64로 마이그레이션
- Read, Write Instance 분리하여 Aurora RDS Postgresql 운영
- Managed 서비스와 EC2 서비스 사용에 대한 리소스 비용을 저울질 하여 서비스 취사 선택
- **리소스 별 적절한 권한 부여**

MLOps Engineer, MLOps

Oct 2021 - Jun 2022
중구, 서울

미술시장 분석 PoC

Apr 2022 - Jun 2022

- HuggingFace의 NLP를 이용하여 Entity Recognition 기술을 이용하여 아티스트의 CV 데이터 정량적 분석 및 정제
- Word Histogram 을 이용하여 기사 분석
- Sentiment Analysis 을 이용하여 기사 분석
- NLP를 이용하여 Text 분석

인스타그램 관계 데이터 분석

Feb 2022 - Apr 2022

- Node2Vec, K-means, PCA 기술들을 사용하여 관계 분석
- NextJS, Flask, CytoscapeJS를 이용하여 Directed Graph 관계를 보여주는 데이터 시각화 페이지 개발
- 분석 자료를 토대로 Series A 투자 유치
- ML Model 추론 기능을 사용하기 위한 input 형태로 정제, feature 추출 및 추론 값 분석
- [Project Detail >>](#)

인스타그램 관계 정보 스크래퍼 개발

Oct 2021 - Jun 2023

- 지속적인 데이터 수집을 위한 여러 방법 구상 및 구현
- Selenium 이용한 동적 페이지 스크래퍼 개발
- requests를 이용하여 PublicAPI를 통한 데이터 스크래퍼 개발
- Instagram의 private API를 사용하는 Open Source Project인 Instagrapi 를 이용하여 데이터 스크래퍼 개발
- Python으로 작성된 Instagrapi를 Swift 언어로 마이그레이션 하여 모바일 환경에서 동작하는 데이터 스크래퍼 개발
- 소셜 관계 정보 수집을 위해 OpenSource project에 기여: [Instagrapi](#)
- 도전적인 문제를 해결하기 위한 지속적인 시도
- [Project Detail >>](#)

미술 아티스트 정보 스크래퍼 개발

Oct 2021 - Nov 2021

- requests를 이용해서 정적 페이지 스크래퍼 개발
- boto3와 MultiThreading을 이용하여 다중 EC2 인스턴스 구동으로 고속 스크래핑 환경 개발
- 60만의 아티스트 정보, 300만의 아티스트 CV 정보 수집
- 데이터 분석을 위한 초석 확보

PostgreSQL 기반 수집 데이터 관리

- Schema 분리를 통한 Raw Data와 Purified 데이터 구분하여 보관
- Python과 psycopg2를 이용하여 데이터 정제
- 초기 형태의 데이터 파이프라인 구축
- [Project Detail >>](#)

iOS Developer, iOS Team

Jan 2019 - Oct 2021
강남, 서울

iOS 어플리케이션 개발 및 배포

- Fastlane과 Github Action 을 이용하여 CI/CD 구축
 - ARKit을 이용하여 Artwork를 벽에 걸 수 있는 Artwork Hanger 기능 구현
 - Push Notification 기능 구현
 - Alamofire를 이용하여 Backend의 API를 사용하기 위한 API Layer 구축
 - Unity Bridge를 이용하여 Unity의 GoogleCardboard VR 기능 연동
 - VR 기능을 위해 AWS S3에 저장된 Asset을 다운로드 하는 기능 구현
 - 다중 다운로드 기능을 지원하기 위해 Download Service 구현
 - Reactive Pattern Programming 을 구현하기 위해 ReactorKit을 사용, ReactorKit 사용을 위한 Model, View, ReactorKit 아키텍처 구현
 - 단위 테스트 환경 구축 및 점진적 커버리지 확대
 - SnapKit을 이용하여 Autolayout 구현
 - Kingfisher를 이용하여 ImageView 개발
 - Pinterest Style CollectionView 개발
 - 소규모 iOS 인하우스 어플리케이션 작성 가능
-

(주) Finkle

Jan 2018 - Nov 2018

Software Engineer, DevOps

영등포, 서울

iOS Hybrid Application 유지보수

Jan 2018 - Nov 2018

- VueJS 기반 Web을 사용하여 Alert 기반 통신을 하는 UIWebView, WKWebView 를 사용하는 iOS App 유지보수
- Objective-C -> Swift 마이그레이션
- Alert으로 전달되는 ActionCode에 해당하는 기능 개발

Android Hybrid Application 유지보수

Jan 2018 - Nov 2018

- VueJS 기반 Web을 사용하여 Alert 기반 통신을 하는 ChromeWebView Android App 유지보수
- Java를 사용하여 유지보수
- Alert으로 전달되는 ActionCode에 해당하는 기능 개발

NodeJS 기반 데이터 스크래퍼 개발

Jan 2018 - Nov 2018

- 기존 작성된 암호화 모듈을 이용하여 Login하는 기능 구현
- 특정 페이지를 Pagination 하여 스크래핑 하는 기능 구현

Education

Coursera

Feb 2023

Convolutional Neural Networks

Coursera

Nov 2022

Structuring Machine Learning Projects

Coursera

Oct 2022

Improving Deep Neural Networks: Hyperparameter Tuning, Regularization and Optimization

Coursera

Sep 2022

Neural Networks and Deep Learning

정보처리기사

Nov 24 2017

한국산업인력공단: 17203100440V

정보처리운용기능사

Jul 20 2009

한국산업인력공단: 09403054754L

언어

- English: AL (OPic)
- Japanese: 일상 회화 가능
- Korean: Native

한국공학대학교

Mar 2010 - Feb 2018

학사, 컴퓨터 공학 (GPA: 3.4 / 4.5) (GPA in Major: 3.73 / 4.5)

시흥, 경기도

- **Relevant Coursework:** Data Structures, Algorithms (C), Operating System, Computer Architecture, Clouding Computing
- [프로보노] 사회적 약자를 위한 증강현실을 이용한 가구배치 시스템

Others

구룡마을 연탄봉사

Jan 2018

Let's Us go iOS Hackathon 참가

Jun 2024

OpenAI API를 이용하여 음식 레시피를 정형화된 응답으로 받을 수 있는 Layer 개발

대한민국 육군 하사 만기제대 (전문하사)

Jun 2011 - Sep 2013

Visual Basic Macro를 이용한 xlsx 임금계산기 작성

베어메탈 K8s 위의 In-House MLOps 플랫폼 설계 & 운영 (Pulumi IaC)

- **핵심 문제 및 목표:** ML 연구자가 손쉽게 분산 학습을 submit 하고, 데이터 엔지니어가 lakehouse / 분석 워크로드를 안정적으로 운영할 수 있는 **사내 전용 MLOps 플랫폼**을 베어메탈 자원 위에 처음부터 구축해야 했음. 외부 매니지드 서비스 의존 없이 **저장 · 카탈로그 · 계보 · 오케스트레이션 · 분산 학습 · 관측까지 풀스택**을 단일 클러스터에서 일관되게 운영하는 것이 목표.
- **나의 역할:** 다중 노드(GPU · 스토리지 · CPU 분리) kubernetes 클러스터 위에 **전 스택**을 단일 담당으로 설계 · 구축 · 운영. IaC, 네트워크, HA, lakehouse, 오케스트레이션, 분산 학습, 관측 모두 포함.
- **문제 해결 과정:**
 - * **트레이드오프 분석 (Managed vs. Self-hosted):** 매니지드 K8s / 매니지드 lakehouse 는 빠른 출발이 가능하지만, 사내 자원 · 보안 요구사항 및 장기 운영 비용을 감안하면 **베어메탈 self-hosted 가 우월**하다고 판단. 초기 구축 부담을 감수하고 모든 layer 를 in-cluster 로 가져감.
 - * **구체적인 구현 내용 (How):** Pulumi(Python) 로 인프라 코드를 12개 stack(platform → base → storage → catalog → lineage → orchestration → staging → compute → mlops → cicd → identity → devenv)으로 모듈화. 단일 명령으로 reproducible deploy 가 가능하도록 stack 간 reference 와 dependency 를 명시적으로 관리. Cilium CNI(KPR + Gateway API + LB-IPAM + L2 Announcements) 기반 네트워크 구성, 단일 Gateway 로 *.<internal>.* 호스트명을 자체 CA HTTPS 로 일괄 노출.
- **결과 및 임팩트:**
 - * **재현 가능한 IaC 체계 정착:** 인프라 변경이 PR 기반으로 추적 가능하고, 신규 환경(예: 별도 staging) 도 stack 조합으로 즉시 재구성 가능.
 - * **운영 자산화:** 모든 호스트명이 단일 Gateway 로 노출되어 인증서 / 라우팅 관리가 일원화되고, 클러스터 외부 의존성을 최소화.

SPoF 제거를 위한 HA 아키텍처 설계 (CoreDNS, Airflow scheduler Active-Active)

- **핵심 문제 및 목표:** 단일 노드 사고가 발생해도 **학습 데이터 파이프라인이 정지하지 않아야** 했음. 클러스터 내 핵심 컴포넌트(CoreDNS, Airflow scheduler)가 단일 인스턴스로만 운영되면 노드 장애 시 전체 워크로드가 중단되는 구조였음.
- **나의 역할:** SPoF 후보를 식별하고, 컴포넌트별로 **multi-node active-active 이중화 정책**을 설계 & 도입. 운영 절차를 runbook 으로 문서화.
- **문제 해결 과정:**
 - * **기술적 고민 및 선택 (Why):** Active-passive 페일오버는 detection / failover lag 가 길어 짧은 다운타임을 피할 수 없음. 학습 데이터 파이프라인 특성상 **재시작 후 catch-up 비용이 크기 때문에**, 다소 운영 복잡도가 올라가더라도 **active-active 다중 인스턴스**를 선택.
 - * **구체적인 구현 내용 (How):** CoreDNS · Airflow scheduler 등 SPoF 컴포넌트를 multi-replica 로 배포하고, PriorityClass / PodDisruptionBudget(PDB) 정책을 클러스터 전체에 적용하여 노드 drain / upgrade 시점에도 최소 1개 replica 가 항상 살아있도록 보장. 운영 절차(drain 순서, scheduler 재시작 시 점검 항목, lakehouse 정합성 체크)를 runbook 으로 문서화하여 다음 담당자가 동일 상황을 즉시 follow 가능하게 함.
- **결과 및 임팩트:**
 - * **단일 노드 장애 → 무중단 운영:** 단일 워커 노드 사고 / 정기 upgrade 시에도 학습 DAG 와 dataset 동기화가 중단 없이 진행됨.
 - * **운영 지식의 자산화:** runbook 정착으로 인계 / 협업 시 학습 곡선이 짧아지고, 동일 사고 재발 시 즉시 대응 가능한 체계 마련.

SeaweedFS + Apache Iceberg Lakehouse 구축 (Bronze/Silver/Gold 메달리온)

- **핵심 문제 및 목표:** 학습 데이터 / 분석 데이터의 raw 적재 · 정제 · 서빙을 단일 lakehouse 에서 일관되게 관리해야 했음. 외부 클라우드 의존 없이 **온프레미스에서 S3 호환 분산 스토리지 + 테이블 포맷**으로 lakehouse 를 구성하는 것이 목표.
- **나의 역할:** 스토리지 layer 선정 / 구축, Iceberg 메달리온 아키텍처 설계, ad-hoc 쿼리 layer 도입까지 **lakehouse 전체 책임**.
- **문제 해결 과정:**
 - * **트레이드오프 분석 (MinIO vs. SeaweedFS):** 두 대안 모두 S3 호환이지만, hot/cold data 를 tier 별로 효율 관리해야 하는 요구에 따라 NVMe / HDD 디스크 tier 분리와 multi-node replica volume group 구성이 자연스러운 **SeaweedFS** 를 선택.
 - * **구체적인 구현 내용 (How):** SeaweedFS 위에 Apache Iceberg 를 올려 **Bronze / Silver / Gold 메달리온** 아키텍처를 production 운영. ad-hoc 분석 needs 를 만족시키기 위해 PostgreSQL + pg_duckdb 로 Iceberg 데이터를 직접 SQL 조회 가능하게 구성하고, 분석가용 DuckDB UI 를 클러스터 내 Pod 로 배포(nginx basic auth + 자동 token inject) — 기존에 pyiceberg 코드를 작성해야 했던 단계를 브라우저 SQL 한 줄로 단축.

- 결과 및 임팩트:

- * 분석 사이클 단축: 분석가가 더 이상 pyiceberg / Spark 환경 세팅 없이 브라우저에서 즉시 ad-hoc SQL 실행 가능.
- * 스토리지 비용 효율화: hot data 는 NVMe, 보관용 cold data 는 HDD tier 로 자동 배치되어 단일 클러스터에서 비용 / 성능 균형을 맞춤.

Airflow 3.x + dbt + Cosmos 기반 오케스트레이션 / 변환 파이프라인

- 핵심 문제 및 목표: Silver / Gold 변환과 Iceberg compaction · 정합성 검증을 표준화된 워크플로우로 운영해야 했음. dbt 모델을 Airflow 위에서 자연스럽게 실행하고, prod / staging 환경을 분리하여 릴리즈 위험 없이 마이너 / 패치 업그레이드를 적용하는 체계 구축이 목표.
- 나의 역할: Airflow Helm 배포, 커스텀 이미지 구성, dbt + Cosmos 통합, 환경 분리, 업그레이드 표준화 전 과정 담당.
- 문제 해결 과정:
 - * 기술적 고민 및 선택 (Why): KubernetesExecutor 는 task 단위로 pod 격리가 가능해 리소스 관리와 의존성 충돌 회피에 유리. 사내 dbt 프로젝트 + 사내 SDK 를 같은 이미지에 번들링하여 모든 task 가 동일한 의존성 그래프로 동작하도록 구성.
 - * 구체적인 구현 내용 (How): Airflow 3.x 를 Helm + KubernetesExecutor 로 prod / staging 분리 배포, git-sync 기반 CD + 커스텀 이미지 (dbt + Astronomer Cosmos + 사내 SDK) 운영. dbt + Cosmos 로 Silver / Gold 변환을 TaskGroup 단위 통합, Iceberg compaction · 정합성 검증 DAG 자동화. 모든 변경은 staging-first → prod 순차 적용 패턴으로 표준화.
- 결과 및 임팩트:
 - * 변환 파이프라인 자동화: 데이터 정합성 / compaction 등 운영 task 가 DAG 화되어 수동 작업 / 실수 가능성이 사라짐.
 - * 안전한 업그레이드 체계: staging 검증 후 prod 적용 패턴으로 Airflow / dbt / 의존성 마이너 업그레이드를 무중단으로 진행 가능.

KubeRay 기반 비대칭 GPU 노드 분산 학습 환경

- 핵심 문제 및 목표: 사내에 서로 다른 SKU 의 GPU 노드들이 혼재된 상태에서, ML 연구자가 환경 세팅에 시간을 쓰지 않고 분산 학습 작업을 손쉽게 submit 가능하도록 표준화된 학습 환경을 제공해야 했음.
- 나의 역할: Ray cluster 구성, 학습 이미지 일원화, GPU type 별 worker group 정책 설계, 사내 학습 SDK 자동 설치 파이프라인 구성.
- 문제 해결 과정:
 - * 기술적 고민 및 선택 (Why): 노드별로 환경을 직접 세팅하게 두면 의존성 / CUDA 버전 충돌이 반복 발생. 단일 worker 이미지로 환경을 일원화하고 GPU type 만 worker group 단위로 분리하는 방식이 운영 비용과 사용자 학습 곡선을 모두 줄임.
 - * 구체적인 구현 내용 (How): KubeRay 로 비대칭 GPU 노드 위에 Ray cluster 구성, 단일 worker 이미지 (CUDA 12.8 · Python 3.12 · PyTorch · ONNXRuntime · Triton)로 학습 환경 일원화. GPU type 별 worker group 분리 + 사내 학습 SDK install 자동화로 ML 연구자가 코드 변경 없이 분산 학습을 submit 가능하게 함.
- 결과 및 임팩트:
 - * 환경 세팅 비용 제거: 분산 학습 시작까지 걸리던 환경 세팅 단계가 사라지고, 연구자가 코드 작성에 집중 가능.
 - * 비대칭 자원 활용: 서로 다른 GPU SKU 를 단일 Ray cluster 안에서 함께 활용해 클러스터 자원 활용률 향상.

DataHub / MLflow / Prometheus 통합 카탈로그 & 관측 체계

- 핵심 문제 및 목표: dataset 카탈로그, ML 실험 / 모델 레지스트리, 클러스터 메트릭이 모두 분산되어 있어 한 곳에서 데이터 흐름과 모델 lineage 를 추적할 수 있는 통합 관측 체계가 필요했음.
 - 나의 역할: DataHub 배포 & OpenLineage 연동, MLflow 운영, Prometheus / Grafana 클러스터 메트릭 표준화, self-hosted Docker Registry + GitHub Actions CI/CD 구성.
 - 구체적인 구현 내용 (How): DataHub 로 dataset 카탈로그 관리 + OpenLineage 기반 Airflow / dbt 계보 자동 수집. MLflow 로 실험 / 모델 레지스트리 운영. Prometheus + Grafana 로 클러스터 메트릭 표준화. Self-hosted Docker Registry + GitHub Actions 로 image build · deploy 자동화.
 - 결과 및 임팩트:
 - * 데이터 & 모델 lineage 가시화: dataset 부터 모델까지 흐름을 한 화면에서 추적 가능, 변경 영향 분석이 수동 추적에서 카탈로그 검색으로 단축.
 - * 이미지 공급망 단순화: 자체 Registry + GitHub Actions 로 인프라 / 어플리케이션 이미지 build · deploy 가 단일 워크플로우로 통합.
-

미술품 투자 분석을 위한 통합 데이터 플랫폼 구축

- **핵심 문제 및 목표:** 작가, 작품, 경매, 소셜 네트워크, 기관, 아티스트 CV 등 미술 시장의 모든 데이터가 여러 소스에 파편화되어 있었음. 이 데이터들을 포괄하는 신뢰도 높은 통합 데이터 파이프라인을 구축하고, 이를 AI 및 통계 기반의 데이터 분석 플랫폼으로 활용하여 Series B 투자 유치에 기여하는 것을 최종 목표로 함.
- **나의 역할:** 데이터 엔지니어링 팀 리더로서, 데이터 파이프라인의 아키텍처 설계부터 구현, 운영, LLM 에이전트 프로토타입 개발까지 프로젝트 전반을 주도함.
- **문제 해결 과정:**
 - * **트레이드오프 분석 (Pandas vs. PySpark):** 프로젝트의 최종 목표는 모든 데이터를 수집하는 것이었기에, 미래에 데이터 규모가 수억 건 이상으로 커질 가능성이 높다고 가정하고 아키텍처를 설계함. 현재의 빠른 개발보다 미래의 확장성(Scalability)과 신뢰성(Reliability)을 확보하기 위해, 초기 오버헤드를 감수하고 선제적으로 PySpark 도입을 결정함.
 - * **구체적인 구현 내용 (How):** AWS EMR 클러스터가 생성될 때, 각각의 데이터 처리 단계를 정의한 Python 파일들을 EMR Step으로 순차적으로 포함하여 실행하는 방식으로 데이터 정제 및 적재 파이프라인을 구현함. 이 파이프라인 내에서 Facebook의 SS2D 및 OpenAI Embedding 모델을 활용하고, PGVector의 HNSW 인덱스를 사용하여 중복 데이터를 필터링하는 시스템을 구축함.
- **결과 및 임팩트:**
 - * **비즈니스 임팩트:** 구축된 고품질의 통합 데이터 플랫폼은 회사의 핵심 자산이 되었고, Series B 투자 유치를 성공시키는 데 결정적인 역할을 함.
 - * **기술적 성과:** PySpark를 선제적으로 도입하여, 데이터가 급증했을 때 별도의 마이그레이션 없이 안정적으로 파이프라인을 운영할 수 있는 확장 가능한 아키텍처의 기반을 마련함.

[go back](#)**대규모 데이터 스크래퍼 중앙 관리 및 모니터링 서버 개발**

- **핵심 문제 및 목표:** 미술 시장 데이터 수집을 위해 동시에 50개 이상의 스크래퍼 인스턴스를 운영하게 되면서, 각 인스턴스의 상태를 개별적으로 추적하고 관리하는 리소스 소모가 컸음. 여러 스크래퍼를 중앙에서 제어하고 실시간으로 모니터링하여 데이터 수집 파이프라인의 안정성과 운영 효율성을 극대화하는 것을 목표로 함.
- **나의 역할:** 서버의 전체 아키텍처 설계부터 API 엔드포인트 개발, WebSocket 통신, Nginx/HTTPS 설정 및 배포까지 End-to-End 개발을 모두 담당함.
- **문제 해결 과정:**
 - * **기술적 고민 및 선택 (Why):** 실시간 상태 모니터링을 위해, 클라이언트가 주기적으로 서버 상태를 요청하는 HTTP Polling 방식보다 서버와 클라이언트 간의 양방향 통신 채널을 유지하는 WebSocket 방식이 서버 부하가 적고 상태 변화를 즉각적으로 전파할 수 있어 더 효율적이라고 판단함.
 - * **트레이드오프 분석 (Trade-off):** 경량 웹 프레임워크인 'Flask'와 'FastAPI'를 두고 고민함. 당시 프로젝트의 핵심은 안정적인 실시간 통신 구현이었기에, 관련 라이브러리 생태계가 성숙하고 제가 더 익숙했던 'Flask'를 선택하여 개발 속도와 안정성을 우선으로 확보함.
 - * **구체적인 구현 내용 (How):** 'Flask'를 기반으로 스크래퍼 제어 API를 개발하고 'Flask-Blueprint'로 코드를 모듈화함. 'WebSocket'을 통해 각 스크래퍼와 연결을 유지하며 상태 및 메트릭을 실시간으로 교환하도록 구현함. 보안을 위해 로그인 정보는 'SHA-256'으로 암호화 하고, 'Nginx'를 리버스 프록시로 사용하여 'Certbot'으로 발급받은 HTTPS 인증서를 적용함.
- **결과 및 임팩트:**
 - * **운영 효율성 극대화:** 수동으로 관리하던 50개 이상의 스크래퍼 인스턴스를 단일 API를 통해 중앙에서 관리할 수 있게 되어, 운영 리소스가 획기적으로 감소함.
 - * **안정성 및 신뢰성 향상:** 장애 발생 시 특정 인스턴스를 즉각적으로 파악하고 원격으로 재시작할 수 있게 되어, 데이터 수집 파이프라인 전체의 안정성과 데이터 누락 방지 신뢰성이 크게 향상됨.

[go back](#)**소셜 관계망 분석 및 데이터 정제를 위한 Full-Stack BI 툴 개발**

- **핵심 문제 및 목표:** ML 모델이 분석한 소셜 관계망 데이터는 복잡한 그래프 형태로, 비개발자인 미술 전문가들이 직관적으로 이해하고 분석하기 어려웠음. 전문가가 직접 데이터를 검토하고 수정(라벨링)하여 데이터 품질을 개선하는 효율적인 도구가 필요했음.
- **나의 역할:** 데이터 구조 설계, API 개발(Flask), 프론트엔드(Next.js) 및 시각화(Cytoscape/SigmaJS) 구현, 그리고 최종 배포(Docker, Nginx)까지 전체 Full-Stack 개발을 주도함.
- **문제 해결 과정:**
 - * **기술적 고민 및 선택 (Why):** 대규모 그래프 데이터를 브라우저에서 원활하게 렌더링하는 것이 핵심 과제였음. 초기에는 Cytoscape.js를 사용했으나, 노드와 엣지가 늘어나자 성능 저하가 발생하여 WebGL 기반의 Sigma.js로 마이그레이션하여 성능을 개선함.

- * **트레이드오프 분석 (Trade-off):** UI 컴포넌트 개발 시, 범용 라이브러리 대신 IBM의 **Carbon Design System** 을 선택함. 국내 자료가 적어 초기 학습이 필요했지만, 데이터 시각화와 대시보드에 특화된 컴포넌트가 많아 장기적인 개발 생산성을 높일 수 있다고 판단함.
 - * **구체적인 구현 내용 (How):** Flask 로 소셜 관계 데이터와 데이터 라벨링을 위한 RESTful API를 개발함. **Next.js**와 **Carbon Design System** 으로 프론트엔드 환경 및 UI 컴포넌트를 구현함. 최종적으로 **Docker Compose** 를 이용해 Nginx, Next.js, Flask 서버를 컨테이너화하여 EC2에 배포함.
 - **결과 및 임팩트:**
 - * **업무 효율성 향상:** 미술 전문가들이 DB 조회나 개발자 요청 없이, 직접 BI 툴을 통해 소셜 관계망 데이터를 탐색하고 분석할 수 있게 되어 업무 시간이 획기적으로 단축됨.
 - * **데이터 품질 개선:** 전문가가 시각화된 그래프를 보며 직접 데이터 오류를 수정하고 라벨링하는 기능을 제공하여, ML 모델의 학습 데이터 품질을 높이는 선순환 구조의 기반을 마련함.
- [go back](#)

비용 효율적인 데이터 파이프라인 자동화를 위한 Airflow 구축 및 운영

- **핵심 문제 및 목표:** 데이터 수집과 정제 파이프라인이 수동으로 운영되어 비효율적이고 오류 발생 가능성이 높았음. 전체 데이터 파이프라인을 워크플로우로 정의하여 **실행, 모니터링, 재처리**를 자동화하고, **운영 비용을 최소화**하는 안정적인 스케줄링 시스템 구축을 목표로 함.
 - **나의 역할:** Airflow 인프라 설계, EC2/Docker 기반 배포, 핵심 DAG(데이터 수집, EMR 처리) 작성 및 운영까지 워크플로우 자동화의 전 과정을 담당함.
 - **문제 해결 과정:**
 - * **트레이드오프 분석 1 (인프라):** AWS Managed Airflow(MWAA)는 높은 비용 외에도, 당시 제가 선호하는 **TaskFlow API**를 사용할 수 없는 버전상의 제약이 있었음. 최신 버전을 자유롭게 사용하고 인프라 비용을 90% 이상 절감하기 위해, 초기 구축 리소스가 더 필요하더라도 **EC2에 Docker Compose로 직접 배포**하는 방식을 선택함.
 - * **트레이드오프 분석 2 (Executor):** 병렬 처리를 위한 **Celery Executor**는 추가 인프라가 필요해 복잡성이 높다고 판단. 당시 Task의 순차적 안정성이 더 중요했기에, 가장 단순하고 안정적인 **Sequential Executor**를 선택하여 인프라 복잡도를 최소화함.
 - * **구체적인 구현 내용 (How):** **Docker Compose**를 이용해 Airflow를 EC2에 배포. 스크래핑 DAG는 **PythonOperator**를 사용하여, 직접 로직을 실행하는 것이 아니라 외부의 **스크래퍼 중앙 관리 서버의 API**를 호출하여 작업을 스케줄링하는 방식으로 작성함. 데이터 정제 DAG는 **boto3**로 EMR 잡(Job)을 트리거하도록 구현함.
 - **결과 및 임팩트:**
 - * **운영 효율성 극대화:** 매일 수동으로 실행하던 데이터 작업을 **완전 자동화**하여, 관련 운영에 소요되던 시간을 95% 이상 단축함.
 - * **안정성 및 관측 가능성 향상:** Airflow UI를 통해 모든 파이프라인의 실행 상태와 로그를 중앙에서 모니터링할 수 있게 되어, 장애 발생 시 원인 파악 및 대응 시간이 크게 단축됨.
- [go back](#)

임베딩 기반 이미지/텍스트 중복 제거 시스템 개발

- **핵심 문제 및 목표:** 초기 중복 제거 시스템은 DB에서 임베딩을 다운로드하고, 데이터 타입을 변환한 후, 외부에서 인덱스를 생성하는 **불필요한 오버헤드**가 발생. 이 비효율을 제거하고 DB 내에서 모든 로직을 처리하여 **파이프라인을 최적화**하고 데이터 품질을 높이는 것을 목표로 함.
 - **나의 역할:** ML 엔지니어가 개발한 초기 시스템의 파이프라인 오버헤드를 분석하고, **PGVector** 기반의 새로운 아키텍처를 설계 및 도입하여 최적화하는 작업을 주도함.
 - **문제 해결 과정:**
 - * **기술적 고민 (기존 방식의 문제점):** 초기 'Faiss-CPU' 기반 시스템은 PostgreSQL에서 데이터를 꺼내고, **호환되지 않는 데이터 타입을 변환**한 후, 로컬에서 인덱스를 생성해야 하는 비효율적인 구조였음.
 - * **구체적인 구현 내용 (How):** 이 오버헤드를 해결하기 위해 모든 로직을 DB 내부에서 처리하도록 변경. 기존 'Numpy' 배열을 'pgvector::vector' 타입으로 마이그레이션하고, 'PGVector'의 'IVFFlat' 및 'HNSW' 인덱스를 생성하여 시스템을 재구축함.
 - **결과 및 임팩트:**
 - * **운영 효율성 및 비용 절감:** 불필요한 데이터 전송 및 변환 과정을 완전히 제거하여, **전체 파이프라인의 오버헤드를 50% 이상 절감**하고 구조를 단순화함.
 - * **데이터 품질 향상:** 최적화된 시스템을 통해 데이터의 정합성과 신뢰도를 획기적으로 향상시켜, 고품질의 Price Database 구축에 핵심적으로 기여함.
- [go back](#)

비용 최적화 및 데이터 신뢰도 향상을 위한 데이터 웨어하우스 재설계

- **핵심 문제 및 목표:** 기존의 단일 RDS 중심 아키텍처는 데이터 **중복** 및 **불필요한 비용** 문제가 심각했음. 데이터 처리 단계별 저장소를 분리하고 **SSoT(Single Source of Truth)** 기반의 아키텍처를 도입하여, 데이터 신뢰도 향상과 운영 비용 절감을 목표로 함.

- **나의 역할:** 데이터 파이프라인의 문제점을 분석하고, 비용 절감 및 신뢰도 향상을 위한 데이터 웨어하우스 구조 재설계를 주도함.
 - **문제 해결 과정:**
 - * **트레이드오프 분석 (RDS vs. S3 Data Lake):** 기존 RDS 구조는 초기 개발이 빠른 장점이 있으나, 장기적인 비용 및 확장성 문제가 명확했음. 초기 마이그레이션 비용을 감수하더라도, 저렴한 S3를 데이터 레이크로 도입하여 장기적인 스토리지 비용 절감 및 관리 유연성을 확보하는 방향으로 결정함.
 - * **구체적인 구현 내용 (How):** DBDiagram 툴을 사용하여 DataSource, Data Warehouse, Data Mart 계층을 시각적으로 설계함. 데이터 중복을 통합 관리하기 위한 SSoT 테이블을 포함하여 약 50개의 테이블 스키마를 설계함.
 - **결과 및 임팩트:**
 - * **비용 절감 기반 마련:** S3 데이터 레이크 도입을 통해 기존 아키텍처 대비 예상 스토리지 비용을 획기적으로 절감할 수 있는 기반을 마련함.
 - * **데이터 신뢰도 향상:** SSoT 테이블 설계를 통해 데이터 중복을 체계적으로 관리하고, 데이터의 정합성과 신뢰도를 높일 수 있는 구조를 확립함.
- [go back](#)

MLOps Engineer, MLOps

Oct 2021 - Jun 2022

중구, 서울

MLOps 기반 인스타그램 관계망 분석 및 투자 인사이트 도출 PoC

MLOps Engineer

- **핵심 문제 및 목표:** 전문가의 정성적 판단에 의존하던 아티스트의 시장 영향력을 인스타그램 관계망 데이터로 객관화/정량화. 데이터 기반으로 성장 가능성을 증명하여, Series A 투자 유치를 성공시키는 것을 최종 목표로 함.
 - **나의 역할:** MLOps 엔지니어로서, 데이터 수집부터 DVC를 이용한 데이터/모델 버저닝, 여러 모델의 입력값 가공, K-Means/PCA 분석 및 결과 시각화까지 PoC의 재현 가능한 End-to-End 파이프라인 구축을 주도함.
 - **문제 해결 과정:**
 - * **기술적 고민 및 선택 (Why):** 당시 머신러닝 모델 자체에 대한 깊은 지식이 부족했기에, ML 리서치 엔지니어와 협업하여 각자의 전문 분야에 집중하는 방식을 선택. 동료는 Node2Vec, GAT 등 다양한 그래프 임베딩 모델링을, 저는 데이터 파이프라인과 후속 분석/시각화에 집중함.
 - * **구체적인 구현 내용 (How):** Git과 연동되는 DVC(Data Version Control)를 도입하여, 모델의 입력 데이터와 산출물을 Git 커밋 해시와 연동하여 버저닝하고 실험의 재현성을 확보함. 이후, 추출한 임베딩 벡터를 입력받아 K-Means 클러스터링과 PCA로 작가 그룹을 분석함. 최종적으로 이 분석 결과를 Next.js 및 Cytoscape.js를 활용하여 상호작용 가능한 네트워크 시각화 대시보드로 구현함.
 - **결과 및 임팩트:**
 - * **비즈니스 임팩트:** 개발된 시각화 대시보드의 분석 자료를 통해 아티스트의 성장 가능성을 증명, Series A 투자 유치를 성공시키는 데 결정적인 기여를 함.
 - * **개인 및 기술적 성장:** 이 협업 경험을 통해 머신러닝 파이프라인 구축에 대한 깊은 실무 이해를 얻었으며, 데이터 엔지니어로서의 커리어로 본격적으로 전환하는 결정적인 계기가 됨.
- [go back](#)

인스타그램 봇 탐지 우회를 위한 데이터 스크래퍼 개발 및 고도화

- **핵심 문제 및 목표:** 인스타그램의 강력한 봇 탐지(Bot Detection) 시스템으로 인해 데이터의 지속적인 수집이 불가능한 문제를 해결. 봇 탐지를 안정적으로 우회하여, 대규모 소셜 관계 데이터를 중단 없이 수집할 수 있는 견고한 스크래퍼 개발을 목표로 함.
 - **나의 역할:** 봇 탐지 문제 해결을 위한 다양한 기술적 접근법(Selenium, requests, Instagrapi 등)을 리서치, 프로토타이핑하고, 최종 솔루션을 구현하는 역할을 담당함.
 - **문제 해결 과정:**
 - * **트레이드오프 분석 (서버 vs. 모바일 환경):** 서버 환경에서 Python 기반의 스크래퍼를 사용하는 것은 결국 봇 탐지에 의해 차단됨을 확인함. '서버 환경' 자체가 탐지의 원인이라는 가설 하에, 초기 비용이 높더라도 Python 라이브러리를 Swift로 직접 마이그레이션하여 모바일 환경 요청을 흉내 내는 근본적인 해결책을 선택함.
 - * **구체적인 구현 내용 (How):** Python으로 작성된 'Instagrapi'의 핵심 로직을 Swift 언어로 직접 포팅하여, iOS 모바일 환경에서 동작하는 데이터 스크래퍼를 개발함. 개발 과정에서 발견한 라이브러리 버그를 수정하여 오픈소스 프로젝트에 직접 기여함.
 - **결과 및 임팩트:**
 - * **핵심 문제 해결:** 기존 방법으로는 불가능했던 대규모 소셜 관계 데이터를 안정적으로 수집하는 데 성공하여, 'MLOps 분석 PoC' 프로젝트의 기반을 마련함.
 - * **문제 해결 역량 입증:** 프레임워크를 다른 언어로 포팅하는 창의적인 접근을 통해, 고질적인 외부 시스템의 제약사항을 해결함.
- [go back](#)

PostgreSQL 스키마 분리를 이용한 초기 데이터 파이프라인 구축

- **핵심 문제 및 목표:** 수집한 원본(Raw) 데이터와 정제된(Purified) 데이터를 효율적으로 분리 및 관리할 필요가 있었음. 별도의 웨어하우스 구축이 어려운 상황에서, 단일 DB 내에서 데이터 무결성을 보존하고 안정적으로 데이터를 제공하는 실용적인 파이프라인 구축을 목표로 함.
 - **나의 역할:** 데이터 정제 로직을 개발하고, PostgreSQL 내에서 스키마를 분리하여 데이터 계층을 나누는 아키텍처를 직접 설계하고 구현함.
 - **문제 해결 과정:**
 - * **트레이드오프 분석 (DB 분리 vs. 스키마 분리):** 물리적인 DB 분리는 비용과 관리 포인트가 2배로 늘어나는 단점이 있었음. 대신, 추가 비용 없이 논리적 분리가 가능한 PostgreSQL의 스키마 분리 방식을 선택하여 실용성과 비용 효율성을 확보함.
 - * **구체적인 구현 내용 (How):** PostgreSQL 내에 raw_data와 purified_data 스키마를 생성. Python의 psycopg2 라이브러리를 사용하여 데이터 정제 로직을 수행하고, 각 스키마의 테이블에 데이터를 저장하는 파이프라인을 구축함.
 - **결과 및 임팩트:**
 - * **데이터 무결성 확보:** 원본 데이터는 변경 없이 보존하고, 분석가는 정제된 데이터만 안전하게 사용할 수 있는 환경을 구축하여 데이터 관리의 복잡성을 낮춤.
 - * **성장의 발판 마련:** 이 경험을 통해 데이터 웨어하우스의 필요성을 깨닫고, 이후 더 고도화된 데이터 아키텍처를 설계하는 역량의 기반을 다짐.
- [go back](#)

iOS Developer, iOS Team

Jan 2019 - Oct 2021

강남, 서울

iOS 어플리케이션 개발 및 배포

iOS Developer

- **핵심 문제 및 목표:** 미술 관련 콘텐츠를 네이티브 앱에서
- **나의 역할:** 네이티브 iOS 어플리케이션의 API Client를 포함한 Backend Layer를 주도하여 구축. 작품, 작가, 전시 상세 View 및 ViewController 구현, Unity Bridge 를 이용하여 Google Cardboard VR 기능 구현, Google Cardboard VR 기능을 위한 Asset들을 다운로드 할 수 있도록 Download 기능 구현, ARKit을 이용하여 Artwork Hanger 기능 구현
- **문제 해결 과정:**
 - * **기술적 고민 및 선택 (Why):** VR 에셋 다운로드, Unity 연동 등 복잡한 기능 추가를 앞두고, 기존 MVC 구조의 복잡성 증가 및 유지보수성 저하를 예상. 상태(State)를 보다 명확하고 예측 가능하게 관리할 수 있는 반응형 아키텍처 도입을 목표로 함.
 - * **트레이드오프 분석 (Trade-off):** 직접 MVVM 패턴을 구현하는 방안을 먼저 고려함. 하지만 이는 상태와 사용자 입력을 처리하기 위한 프로토콜 정의, 데이터 바인딩 로직 구현 등 상당한 초기 개발 리소스가 필요하다고 판단. 대안으로, 반응형 프로그래밍 프레임워크인 ReactorKit을 검토함. ReactorKit은 사용자 입력(Action) -> 비즈니스 로직 처리(Mutate) -> 상태 변경 적용(Reduce) 으로 이어지는 3단계의 명확한 데이터 흐름을 강제하여, 직접 구현 시 발생할 수 있는 복잡성을 원천적으로 차단하고 코드의 흐름을 매우 쉽게 이해할 수 있도록 만들어주는 점이 인상적이었음. 초기 학습 곡선을 감수하더라도, ReactorKit을 도입하는 것이 장기적인 유지보수성과 개발 효율성 측면에서 더 유리하다고 결론 내림.
 - * **구체적인 구현 내용 (How):** ReactorKit을 기반으로 앱의 아키텍처를 재설계. 모든 사용자 입력은 'Action'으로, 상태 변화는 'State'로 정의하여 데이터 흐름을 통일하고, AWS S3 클라이언트를 이용한 다운로드 서비스를 Reactor에 통합하여 구현.
- **결과 및 임팩트:**
 - * **개발 효율성 향상:** 프레임워크에 익숙해진 후, Action-Mutate-Reduce 구조에 맞춰 로직을 정의하는 과정이 매우 쉬워지고 명확해짐. 이로 인해 신규 기능 추가 시 개발 속도가 향상되고 코드의 일관성이 유지됨.
 - * **안정성 및 유지보수성 확보:** 명확한 구조 덕분에 상태 관리 관련 버그 발생률이 감소했으며, 새로운 팀원이 코드 구조를 파악하는 시간이 단축됨.